# Applied AI Assignment

### AI Support Triage and Knowledge Retrieval API

*Python | FastAPI | SQL | NLP | AI Model Integration | Docker | GitHub*

## 1. Objective

Build a backend AI service that accepts a support ticket, analyzes the text, recommends a category and priority, retrieves relevant knowledge base articles, invokes an AI model for an assisted output, and stores the full result in a database.

This is a 3 day individual assessment. The focus is code structure, API design, NLP logic, model integration, persistence, Docker setup, testing, and daily GitHub discipline.

## 2. Problem Statement

Create an AI Support Triage API for an enterprise support team. The system receives a new support ticket and returns a structured recommendation to help a support agent decide the next action.

The application must not only classify the ticket. It must combine text processing, similarity search, historical ticket comparison, knowledge base retrieval, model assisted output, and database persistence.

## 3. Required Technology Scope

| Area | Expectation |
|---|---|
| Python | Core logic, text processing, scoring, model provider integration |
| FastAPI | REST APIs, Pydantic schemas, routing, Swagger docs |
| SQL database | Persist articles, historical tickets, requests, recommendations, audit fields |
| NLP | Preprocessing, TF-IDF or embeddings, cosine similarity, keyword signals |
| AI model | One free or local model provider must be integrated |
| Docker | Dockerfile and Docker Compose with backend and database |
| GitHub | Daily commits, README, basic CI workflow |
| Testing | Pytest tests for core logic and API routes |

## 4. Data Pack Provided

A data pack is provided with knowledge base articles, historical resolved tickets, sample input tickets, payload examples, and model provider notes. Use this data for seeding and local testing.

- Do not spend assessment time creating your own baseline data.
- You must design the schema and ingestion approach yourself.
- You may add more data if required, but the provided data must be supported.
- Do not hardcode results for the provided sample tickets.

| File | Purpose |
|---|---|
| kb_articles.json / csv | Published knowledge base articles used for retrieval |
| historical_tickets.csv | Resolved tickets with category, priority, and resolution summary |
| evaluation_tickets_input.csv | Sample incoming tickets for local demo and validation |
| sample_api_payloads.json | Example input and minimum output fields |
| model_provider_options.md | Allowed model integration options |
| .env.example | Environment variable baseline |

## 5. Functional Requirements

### 5.1 Ticket Analysis

- Accept a ticket title and description through an API request.
- Clean and preprocess the input text.
- Recommend one category from the supported categories in the data pack.
- Recommend one priority: Low, Medium, High, or Critical.
- Return a confidence score between 0 and 1.
- Return a short reasoning summary explaining the recommendation.

### 5.2 Retrieval

- Return the top matching knowledge base articles.
- Return the top similar historical tickets.
- Use TF-IDF plus cosine similarity or embeddings plus cosine similarity.
- Similarity logic must be implemented in a service layer.

### 5.3 AI Model Integration

- Integrate at least one AI model provider or local model runtime.
- Use the model for one meaningful task: triage validation, response draft, summary, or category check.
- Model settings and keys must come from environment variables.
- The API must still work if the model provider is unavailable. Return a fallback result and log the failure.
- Keep model integration in a separate provider or service class.

### 5.4 Persistence and Reporting

- Store each analysis request and result in the database.
- Store selected suggested article ids and similar ticket ids.
- Provide a basic summary endpoint for count by category, count by priority, and latest requests.
- Use audit fields such as created_at and updated_at where relevant.

## 6. API Requirements

The exact request and response schemas must be designed by the trainee. The following endpoints are mandatory.

| Method | Endpoint | Purpose |
|---|---|---|
| GET | /health | Return application and database health |
| POST | /tickets/analyze | Analyze one incoming ticket and persist the result |
| GET | /tickets/history | Return previous analysis records with filters or pagination |
| GET | /kb/search | Search knowledge base articles by query text |
| GET | /analytics/summary | Return basic category and priority summary |
| GET | /model/status | Return configured model provider and availability status |
| GET | /docs | FastAPI Swagger documentation |

Minimum input fields for ticket analysis: title, description, channel, requester_type. Additional fields may be added if justified in README.

## 7. Suggested Application Structure

Use a clean structure. The exact names may differ, but separation of concerns is required.

- app/main.py
- app/routes/
- app/schemas/
- app/models/
- app/services/
- app/repositories/
- app/core/
- tests/
- Dockerfile
- docker-compose.yml
- README.md

## 8. Day Wise Delivery Plan

| Day | Focus | Minimum Deliverables |
|---|---|---|
| Day 1 | API foundation, database, data load | Project scaffold, FastAPI routes, DB connection, schema, seed loader, health endpoint, GitHub push |
| Day 2 | NLP and AI logic | Preprocessing, similarity search, category and priority recommendation, AI model integration, persistence, tests, GitHub push |
| Day 3 | Docker, CI, documentation, final readiness | Dockerfile, Docker Compose, pytest execution, GitHub Actions, README, final cleanup, GitHub push |

## 9. Testing Requirements

Minimum 6 meaningful tests are required.

- Health endpoint test
- Ticket analysis success case
- Invalid input validation case
- Similarity search returns relevant articles
- Priority rule handles Critical urgency
- Model failure fallback does not break the API
- Database persistence test if time permits

## 10. Docker and GitHub Requirements

- Dockerfile must run the FastAPI application.
- Docker Compose must start the backend and database together.
- Environment variables must be injected through .env or Compose configuration.
- GitHub repository must be updated before logging out each day.
- Use meaningful commit messages.
- Add .gitignore. Do not commit virtual environments, cache folders, database dumps, or secrets.
- Add a basic GitHub Actions workflow to run tests or validate the build.

## 11. Critical Points to Take Care Of

- Do not hardcode the output for sample tickets.
- Do not put all logic inside FastAPI route functions.
- Do not expose API keys in code or commits.
- Do not make the application dependent only on an external AI API.
- Handle empty or very short ticket text.
- Keep deterministic logic and model output separate.
- Return consistent API responses and error messages.
- README must explain setup, model provider selected, and how to run tests.

## 12. Final Submission Checklist

- GitHub repository link shared
- Application runs locally
- Docker Compose works
- Database seeding documented
- Swagger docs available
- Required APIs working
- AI model provider integrated or fallback documented
- Tests pass
- README completed
- Final code pushed before submission